

Chapter 3: Methodology

3.1 Introduction

This chapter describes the methodology used to evaluate CARTA's deployment complexity across two execution environments: a Kubernetes-based cloud-native environment and an SLURM-based high-performance computing environment. The purpose of the methodology is to establish a controlled and comparable experimental basis for analyzing how each environment supports the deployment, configuration, operation, and maintenance of CARTA.

Both test beds are organized to support the same core application workflow. Users access CARTA through a browser-facing entry point, authenticate through the CARTA controller, and launch user-specific backend sessions that operate on shared astronomical datasets. In both environments, CephFS serves as the shared persistent storage layer, while MongoDB stores persistent application state for the CARTA controller. This common functional basis enabled comparison of the two deployment approaches under similar application requirements.

To support the complexity analysis, the testbed description is organized using three layers: the infrastructure layer, the application deployment layer, and the operational setup and maintenance layer. The infrastructure layer captures the virtual machine environment, cluster setup, networking, storage, and core platform services. The application deployment layer describes how CARTA, MongoDB, authentication, backend execution, and shared storage access were configured. The operational setup and maintenance layer captures the administrative work required to prepare nodes, start services, expose the application, manage users, and maintain the deployment.

This layered structure allows the comparison to go beyond the final system architecture. It also makes it possible to examine the amount of configuration work, dependency handling, manual intervention, and ongoing administrative effort required by each deployment model.

3.2 Research Design

This study follows a comparative experimental testbed design. Two separate CARTA deployments were implemented: one using Kubernetes and one using SLURM. Each deployment was built to satisfy the same functional requirements, but each used a different orchestration and operational model.

The Kubernetes testbed represents a cloud-native deployment approach. In this environment, application components are described using Kubernetes objects such as Deployments, Services, ConfigMaps, Secrets, PersistentVolumeClaims, Ingress resources, and Role-Based Access Control objects. Kubernetes is responsible for scheduling workloads, managing pod lifecycle, exposing services, and maintaining the desired deployment state.

The SLURM testbed represents a high-performance computing deployment approach. In this environment, the CARTA controller runs on a login or control node, while backend execution is delegated to SLURM-managed compute nodes. The deployment relies on node preparation, configuration files, system services, shared authentication, and administrator-managed startup procedures.

The main objective of this research design is to compare the effort required to deploy and operate CARTA across environments. The comparison focuses on deployment complexity rather than application performance. Complexity is evaluated using measurable indicators such as setup steps, configuration artifacts, lines of configuration, manual intervention, dependency handling, change effort, failure recovery, and operational maintenance effort.

3.3 Overall Testbed Design

Although the Kubernetes and SLURM environments differ in orchestration style, both were designed around the same functional goals. First, each environment needed to support authenticated multi-user access to CARTA. Second, backend sessions had to start in directories specific to the authenticated user, so that filesystem permissions were respected during data access. Third, both environments had to integrate with shared CephFS storage to provide a common location for astronomical datasets and user work directories. Finally, both deployments required MongoDB to store the CARTA controller's application state and to provide a browser-accessible entry point for users.

The common application workflow was therefore as follows. A user connects to the CARTA web interface through a browser-facing access point. The request is routed to the CARTA controller, which handles authentication and session management. After authentication, the controller launches a backend session for the user. The backend then operates on data stored in CephFS, with access controlled through user-specific directories and filesystem permissions.

The main difference between the test beds lies in how these requirements were realized. In the Kubernetes testbed, deployment state was expressed through cluster objects such as Deployments, Services, PersistentVolumeClaims, ConfigMaps, Secrets, Ingress rules, and role-based access controls (RBAC) policies. In the SLURM test bed, the same functional behavior was achieved through node preparation, service installation, configuration files, shared authentication setup, and administrator-managed service startup across the login and compute nodes.

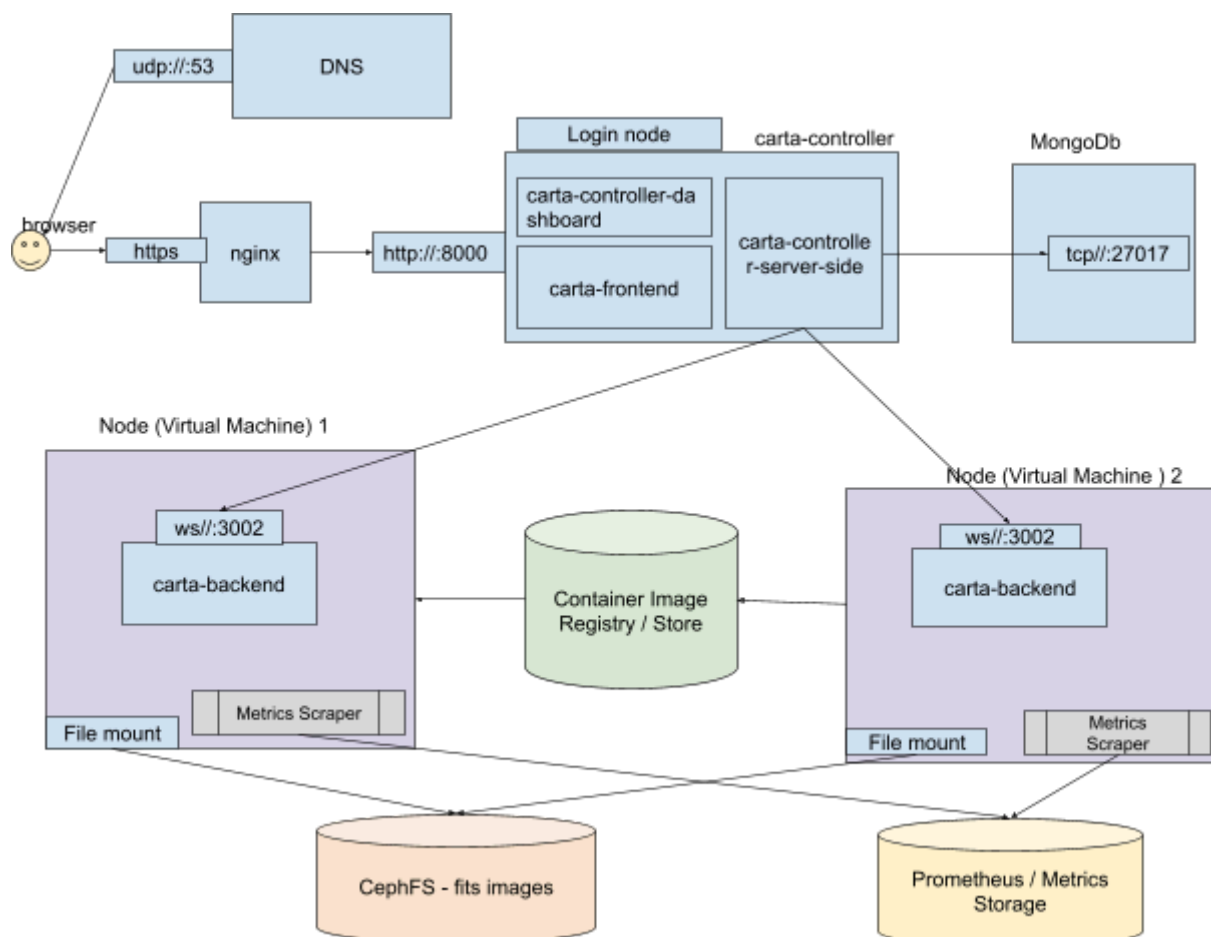
This overall design allowed the two environments to be compared using the same functional baseline. Rather than comparing unrelated deployments, the methodology compares how two different infrastructure models support the same application requirements.

3.4 Architectural Diagram

The two experimental setups follow a comparable high-level architecture built around the same principal services. In both environments, the CARTA controller acts as the user-facing entry point; MongoDB provides persistent application state; CephFS supplies shared persistent storage; and NGINX or an equivalent access mechanism routes external traffic to the controller. Backend execution is separated from the entry layer and placed on compute resources.

Figure 3.1 presents the architectural overview of the two CARTA deployment test beds. The diagram shows the common application components used in both environments, including the CARTA controller, MongoDB, CephFS shared storage, and the browser-facing access layer. It also highlights the main architectural difference between the two deployments: Kubernetes schedules backend instances as pods on worker nodes, whereas SLURM executes backend workloads on compute nodes through its job scheduling model.

Figure 3.1 **High-level CARTA deployment architecture used to compare the Kubernetes and SLURM test beds.**



The key distinction between the two setups is found in the compute layer. In the Kubernetes-based environment, the deployment was hosted on a three-node cluster initialized using kubeadm. The cluster consisted of one control-plane node and two worker nodes. The CARTA controller and supporting Kubernetes services were managed from the

control-plane node, while CARTA backend instances were scheduled onto worker nodes by the Kubernetes control plane.

In the SLURM-based environment, the controller runs on the login or control node, while backend workloads are executed on compute nodes under SLURM scheduling. Instead of creating backend pods, the controller delegates backend execution to the SLURM-managed environment. Thus, while both setups implement the same functional architecture, they differ in how computational resources are provisioned, scheduled, and managed.

The architectural diagram, therefore, supports the comparison by showing that the same application-level requirements were implemented using two contrasting deployment models.

3.5 Virtual Machine Environment

Both test beds were implemented on Ubuntu 22.04 virtual machines provisioned in OpenStack. The use of OpenStack virtual machines provided a controlled infrastructure environment in which both deployment models could be tested under similar conditions.

The Kubernetes environment used three virtual machines on the same subnet. One virtual machine served as the Kubernetes control plane node and was provisioned with 8 GB of RAM, 20 GB of disk storage, and 4 vCPUs. The remaining two virtual machines acted as worker nodes, each provisioned with 32 GB of RAM, 29 GB of disk storage, and 4 vCPUs.

The SLURM environment also used three virtual machines on the same subnet. One virtual machine acted as the login or control node and was provisioned with 8 GB of RAM, 20 GB of disk storage, and 4 vCPUs. The remaining two virtual machines acted as compute nodes, each provisioned with 32 GB of RAM, 29 GB of disk storage, and 4 vCPUs.

The virtual machine layout is summarised in Table 3.1.

Table 3.1: Virtual machine layout for the Kubernetes and SLURM test beds

Component	Kubernetes Test Bed	SLURM Test Bed
Platform	OpenStack	OpenStack
Operating system	Ubuntu 22.04	Ubuntu 22.04
Node 1	Control-plane node: 8 GB RAM, 20 GB disk, 4 vCPUs	Login/control node: 8 GB RAM, 20 GB disk, 4 vCPUs
Node 2	Worker node: 32 GB RAM, 29 GB disk, 4 vCPUs	Compute node: 32 GB RAM, 29 GB disk, 4 vCPUs
Node 3	Worker node: 32 GB RAM, 29 GB disk, 4 vCPUs	Compute node: 32 GB RAM, 29 GB disk, 4 vCPUs

3.6 Kubernetes-Based CARTA Test Bed

3.6.1 Infrastructure Layer

The Kubernetes testbed was deployed using `kubeadm` across three Ubuntu virtual machines. The cluster consisted of one control-plane node and two worker nodes. The control-plane node hosted the Kubernetes control services and provided administrative access through a generated `kubeconfig` file. The worker nodes provided the compute capacity for application workloads, including CARTA backend instances.

Before the cluster could be initialized, each node required operating-system-level preparation. This included installing the container runtime, `kubeadm`, `kubelet`, and `kubectl`. The control-plane node was then initialized using `kubeadm`, and the worker nodes were joined to the cluster using the join command generated during initialization. After cluster formation, a Container Network Interface (CNI) plugin was installed to enable pod-to-pod communication across nodes. This allowed all nodes to transition to the Ready state and support multi-node workload scheduling.

A dedicated Kubernetes namespace named `carta` was created to isolate CARTA-related resources. This namespace contained the CARTA controller deployment, backend-related resources, services, configuration objects, secrets, and access control definitions.

Persistent storage was integrated via the Ceph Container Storage Interface (CSI) plugin. Ceph-CSI exposed external CephFS storage to Kubernetes through storage classes and `PersistentVolumeClaims`. A CephFS-based `PersistentVolumeClaim` using the `ReadWriteMany` access mode enabled multiple pods across different nodes to access the shared file system. This was necessary because CARTA backend sessions needed access to shared astronomical datasets and user-specific directories.

3.6.2 Application Deployment Layer

The CARTA controller was deployed as a Kubernetes Deployment inside the `carta` namespace. The controller configuration was divided between a `Secrets` and a `ConfigMap`. The `Secret` stored sensitive configuration content, including `config.json`, while the `ConfigMap` stored backend runtime parameters.

The controller pod mounted several supporting resources. These included the CARTA authentication key pair, `nsswitch.conf`, and `extrausers` account information. These mounts were required to ensure that user identity resolution inside the container remained consistent with the external filesystem environment. This was important because CARTA

backend sessions needed to access CephFS directories using the correct user and group mappings.

Role-Based Access Control was configured to allow the CARTA controller to manage backend workloads. A service account, role, and role binding were defined so that the controller could create, list, and remove backend pods, as well as inspect pod logs within the namespace. This allowed the controller to dynamically launch user-specific backend instances as Kubernetes pods.

A key application-level modification was introduced to support secure multi-user access. User authentication was handled through PAM, while backend launch arguments were adjusted so that each session opened within a user-specific path derived from the authenticated username. For example, a backend session could be started in a directory such as `/images/${username}`. This ensured that each authenticated user was directed to the correct working directory.

Persistent storage was integrated into the application layer via a CephFS volume mounted on the relevant pods. Together, the controller, PAM authentication, user identity files, RBAC configuration, and CephFS storage ensured that backend sessions were created dynamically while preserving user-specific access control.

A custom controller image was built from a modified Dockerfile to package the required behavior. Supporting platform services included cert-manager for certificate management and MongoDB, deployed via the MongoDB Community Operator. These supporting services formed part of the application deployment layer because they were required for the controller to operate correctly.

3.6.3 Operational Setup and Maintenance Layer

Operationally, the Kubernetes environment required preparation across all three virtual machines before the application could be deployed. This included installing required packages, configuring the container runtime, initializing the control-plane node, joining worker nodes, and installing the cluster networking layer. These steps were necessary before any CARTA-specific manifests could be applied.

After the cluster was operational, Kubernetes resources were applied to deploy and expose the CARTA system. These resources included namespaces, Deployments, Services, ConfigMaps, Secrets, PersistentVolumeClaims, RBAC objects, Ingress resources, and supporting service definitions. The use of manifests allowed the desired state of the deployment to be captured in files that could be version-controlled and reapplied.

To support multi-user access control, user directories were created on CephFS. Ownership was set with the appropriate UIDs and GIDs, and permissions were restricted with `chmod 700` so that each user could access only their own working area. Identity information, including `passwd`, `group`, and `shadow`, was packaged into a Kubernetes Secret and mounted into the controller container together with `nsswitch.conf`. This allowed authentication and filesystem permissions to operate consistently across the application and storage layers.

External access during testing was provided using SSH port forwarding. NGINX was used to support HTTP routing and load balancing for exposure within the OpenStack environment. These components formed part of the Kubernetes operational setup because application access depended not only on the CARTA controller but also on cluster networking, service exposure, and routing configuration.

3.7 SLURM-Based CARTA Test Bed

3.7.1 Infrastructure Layer

The SLURM-based CARTA testbed was deployed on three Ubuntu 22.04 virtual machines provisioned in OpenStack. The cluster consisted of one login/control node and two compute nodes. The login node provided the main administrative and controller-facing environment, while the compute nodes were used to execute workloads scheduled by SLURM. All nodes were placed on the same subnet as a practical design choice to simplify routing, hostname resolution, firewall configuration, and inter-node communication. This ensured that scheduling, authentication, controller-to-compute-node communication, and access to the shared CephFS filesystem worked reliably. The same-subnet configuration was therefore used for simplicity, although the core requirement was network reachability between nodes rather than subnet membership itself.

Infrastructure preparation in the SLURM environment was procedure-oriented. User identity resolution was configured using `extrausers` and modifications to `nsswitch.conf`. User and group identity resolution on managed nodes was configured using `nss_slurm`, where required, allowing SLURM-launched processes to resolve the correct user and group information during job execution.

Authentication between SLURM components was provided by MUNGE. This required creating the `munge` user and group, installing the MUNGE service, configuring secure directory permissions, and distributing a shared `munge.key` across nodes, since MUNGE is required for SLURM daemons to authenticate communication, correct key distribution, and file permissions were essential for cluster operation.

SLURM was installed from source across the cluster. On the login/control node, this involved installing build dependencies, compiling SLURM with PAM support, and configuring supporting services such as MariaDB and `slurmdbd` for accounting. The main configuration files included `slurm.conf`, `slurmdbd.conf`, and `cgroup.conf`. On the compute nodes, the compiled SLURM binaries were also installed, and the required runtime directories, log directories, spool locations, configuration files, and daemon services were prepared manually. The login/control node ran `slurmctld` and `slurmdbd`, while each compute node ran the `slurmd` daemon.

3.7.2 Application Deployment Layer

In the SLURM environment, the application deployment layer focused on adapting CARTA execution to an HPC-oriented workflow. Instead of deploying backends as Kubernetes pods, the CARTA controller delegated backend execution to the SLURM-managed compute environment.

User accounts were created explicitly on the compute nodes. Home directories and ownership settings were configured to ensure users were recognized correctly when backend processes were launched. User-specific directories on CephFS were assigned the correct ownership and restricted using `chmod 700`, ensuring that only the corresponding user could access their own working directory.

The CARTA controller was configured on the login node through a `config.json` file. This configuration defined PAM-based authentication, the MongoDB connection, network binding parameters, backend launch commands, backend kill commands, and folder templates used to determine each user's base directory. The controller, therefore, acted as the user-facing application entry point, while backend execution was delegated to the SLURM scheduling layer.

This model required the controller configuration to align with the SLURM environment. Backend launch commands had to be written to submit or initiate backend processes on the correct compute resources. Folders had to match the CephFS directory structure, and user identity mapping had to remain consistent across the login node, compute nodes, and shared storage.

3.7.3 Operational Setup and Maintenance Layer

Operational maintenance in the SLURM environment depended on explicit administrator actions. Systemd was used to enable and start services such as `munge`, `slurmdbd`, `slurmctld`, and `slurmd`. Each service required correct configuration, file permissions, runtime directories, log paths, and startup order.

Firewall rules were opened for controller, daemon, and database ports. The log, spool file, and PID file locations were created manually. After startup, utilities such as `systemctl status` and `scontrol ping` were used to verify daemon status, control-plane communication, and worker node registration.

During testing, access to the CARTA frontend was provided through an SSH tunnel to the login node, allowing the browser to reach the CARTA controller service without exposing it publicly. In the SLURM test bed, resource management relied on cgroup configuration to constrain CPU, memory, and device access for running workloads. Kubernetes also provides resource controls through Pod and container CPU/memory requests and limits, which are important for preventing uncontrolled resource consumption and managing cluster cost. However, compared with Kubernetes, the SLURM testbed required more direct node-level configuration, repeated setup across machines, and closer coordination among system services.

The operational work in this environment, therefore, included manual preparation of authentication, users, services, directories, firewall rules, and configuration files. These operational tasks were recorded as part of the deployment complexity evaluation.

3.8 Comparative Evaluation Framework

The two test beds were designed to support later deployment-complexity analysis by implementing comparable CARTA application goals through contrasting deployment models. The Kubernetes environment expressed much of its deployment state through declarative cluster objects and controller-driven reconciliation. The SLURM environment implemented the same functional requirements through configuration files, setup scripts, system services, and procedure-driven administration across multiple nodes.

As a result, the test beds provide a basis for comparing deployment complexity in terms of the number of setup steps required, the amount of configuration artifacts involved, the effort needed to express dependencies, the amount of work required to accommodate change, and the extent to which operational behavior is automated or left to the administrator. This makes the test bed appropriate for analyzing deployment complexity using a layered view of infrastructure, application deployment, and operational maintenance.

The comparative framework is summarised in Table 3.6.

Table 3.6: Complexity factors to be evaluated

Aspect	Kubernetes Test Bed	SLURM Test Bed
Execution environment	Kubeadm-based Kubernetes cluster with one control-plane node and two worker nodes	Login/control node plus two compute nodes managed by SLURM daemons
Workload execution	Backend instances are scheduled as Kubernetes pods	Backend processes/jobs executed on SLURM compute nodes
Storage integration	CephFS exposed through Ceph-CSI, storage classes, and PersistentVolumeClaims	CephFS is mounted at the operating-system level on cluster nodes
User isolation	Secrets, extrausers, <code>nsswitch.conf</code> , pod mounts, RBAC, and user-specific start folders	Replicated user accounts, extrausers, CephFS ownership, permissions, and folder templates

Configuration style	YAML manifests and cluster objects	Configuration files, setup scripts, system services, and manual node preparation
Service exposure	Kubernetes Services, Ingress, MetalLB, and SSH port forwarding during testing	SSH tunneling to the login node and controller service exposure
Dependency management	Partly handled by Kubernetes controllers, service discovery, and reconciliation	Managed through installation order, service startup order, and administrator coordination
Failure handling	Pod restart and reconciliation mechanisms are available through Kubernetes	Manual service inspection and restart are often required
Operational style	Cluster administration through kubectl and Kubernetes objects	Procedure-driven administration across multiple nodes
Expected complexity emphasis	Kubernetes objects, RBAC, storage integration, service exposure, and cluster coordination	Node-level setup, repeated configuration, service coordination, and manual maintenance